SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks

Lecture 01

Software Engineering Framework Fundamentals, Selection and Emerging Trends

Year 3 • Semester 1 • 4 Credits

Lecturer: Eishan Weerasinghe | eishan.w@sliit.lk | Course web: courseweb.sliit.lk

Module Delivery Team

Use the correct contact point for module coordination, lecture content, lab support, assessments and special lecture areas.

LIC

Lecture-in-Charge

Eishan Weerasinghe | eishan.w@sliit.lk

VL

Visiting Lecturer

Mr. Lasal Hettiarachchi | lasal.h.visiting@sliit.lk

Official Channels Course Web for announcements and submissions • Email for individual clarification

Teaching & Lab Delivery Team

Centre lecture conductors and module lab conductors for SE3090.

Centre Lecture Conductors

Centre / Group	Lecture Conductor	Email
KANDY UNI (WD/WE)	Ms. Kaushalya Premarathne	kaushalya.pr@kandyuni.lk
NORTHERN UNI (WE)	Ms. A. Sangeetha	sangeetha.b@northernuni.lk

Module Lab Conductors

	Lab Conductor Name(s)	
Ms. Madusha Sulakshi Weerasooriya	Mr. Janidu Illesinghe	Mr. Manuka Rashen
Ms. Piumi Navoda	Ms. Shehani Dehipola	Ms. Hansi De Silva

Lab delivery *Lab conductors support practical completion, environment setup and CourseWeb submission guidance.*

About This Module

- Aim: understand modern frameworks used to build, integrate, deploy and maintain software systems
- Hands-on focus: full-stack web + mobile apps, DevOps and cloud deployment, quality and security
- Responsible use of agentic AI tools in real development workflows

Delivery	Hours	What happens there
Lectures	3 h / week	Concepts, demos, case studies, framework decisions
Laboratories	2 h / week	Guided practicals: React, ASP.NET Core, Flutter, CI/CD, AI tools
Independent study	~80 h / semester	Reading, practice, assignment work

How You Will Be Assessed

10%	15%	25%	50%
Lab Submission	**Mini Hackathon**	**Main Assignment**	**Final Exam**
Continuous lab tasks (start this week)	Participation + Evaluation	Full-stack application with agentic AI integration	Covers all lectures (LO1–LO4)

- Pass requirement: 45% in continuous assessments AND 45% in the final examination
- The main assignment is your chance to apply every framework taught in this module to one product

The Module Journey – 11 Lectures

Foundations	Web Frameworks	Agentic AI	Mobile Frameworks	Quality & Delivery
L01 Framework fundamentals, selection & trends	L02 Advanced React frontend L03 C#, .NET & ASP.NET Core REST APIs L04 Databases, auth & integration	L05–L07 Agentic software development 1–3	L08 Flutter & mobile app development L09 Flutter UI, device features, data	L10 Git, CI/CD, security, code quality L11 DevOps & cloud hosting

Today: the map of the whole territory — every later lecture zooms into one region.

Learning Outcomes for This Lecture

LO1 — Evaluate frameworks

Evaluate different types of software engineering frameworks used in web, mobile, cloud and AI-assisted software development.

LO4 — Select and justify

Select and justify appropriate frameworks, tools and agentic AI-assisted approaches to meet specific project and industry requirements.

By the end of today you should be able to: define what a framework is, distinguish frameworks from libraries, classify common framework types, apply selection criteria to a project scenario, and describe the major emerging trends.

Why Frameworks Matter in Real Software Engineering

Speed to market

Instagram's first version shipped in ~8 weeks on Django. Frameworks remove months of plumbing work.

Built-in quality & security

Mature frameworks ship tested solutions for auth, validation and security — fewer DIY vulnerabilities.

Team scalability

Shared conventions let new developers contribute fast. Industry hires by framework skill: React, .NET, Flutter.

Career reality

Almost every job ad you will apply to names frameworks, not just languages. This module builds that profile.

Today's Roadmap – 3 Hours

Hour 1	**Framework fundamentals** Definitions • why frameworks exist • benefits & costs • frameworks vs libraries • checkpoint
Hour 2	**Types, industry use & selection** Framework landscape • frontend/backend/mobile/testing • industry case examples • selection criteria & workflow
Hour 3	**Trends, case study & wrap-up** Microservices • serverless • cloud-native • DevOps • AI-assisted development • case study + group activity • quiz & summary

Short breaks after each hour • Questions welcome at any point

PART 1 OF 4

Framework Fundamentals

- What a software engineering framework is
- The problems frameworks solve
- Benefits — and the costs nobody advertises
- Frameworks vs libraries (a classic exam question)

What Is a Software Engineering Framework?

Definition: A reusable, semi-complete software platform that provides a standard structure, common services and extension points on which developers build applications.

- **Skeleton of an application — architecture, plumbing and conventions are already in place**
  - You write the parts that make YOUR application unique; the framework runs them
  - Provides ready-made services: routing, data access, UI rendering, security, configuration
  - Examples: React (UI), ASP.NET Core (backend), Flutter (mobile), Spring Boot (Java backend)
- Analogy: a framework is the building's steel frame and utilities — you design the rooms

The Problems Frameworks Solve

Without a framework

- ✕ Re-implement routing, auth, validation, data access for every project
- ✕ Every developer invents a different structure — chaos at handover
- ✕ Security written from scratch → untested, vulnerable
- ✕ Slow starts: weeks of plumbing before the first feature

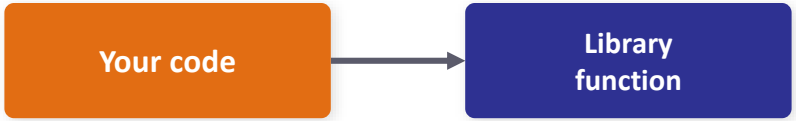
With a framework

- ✓ Common concerns solved once, by experts, and battle-tested
- ✓ One shared structure and naming convention for the whole team
- ✓ Security patches arrive as framework updates
- ✓ Day 1 productivity: scaffold an app in minutes, focus on features

“Don't reinvent the wheel — and definitely don't reinvent the security wheel.”

Inversion of Control – “Don't Call Us, We'll Call You”

Library: YOUR code is in control



You decide when to call lodash, axios, math utils…

Framework: the FRAMEWORK is in control

ASP.NET Core receives the request, then calls your controller. React decides when your component re-renders.

The Hollywood Principle: you plug your code into the framework's extension points; the framework's main loop decides when it runs. This single idea is the cleanest test for 'framework or library?'

Benefits of Using Frameworks

Productivity

Scaffolding, generators, hot reload — features ship faster

Reliability

Battle-tested code paths used by millions of apps

Security

Vetted auth, validation & patching out of the box

Maintainability

Conventions make code predictable and refactorable

Community & talent

Docs, tutorials, Stack Overflow, hireable skills

Best practices built in

MVC, components, DI — good architecture by default

The Costs Nobody Advertises

- Learning curve — Angular or Spring can take weeks before a team is productive
- Lock-in — switching frameworks later usually means a rewrite, not a refactor
- Overhead — abstraction layers can cost performance and bundle size
- “Magic” — hidden behaviour makes debugging harder when conventions are misunderstood
- Upgrade churn — major versions can break code (Angular 1→2, Python 2→3 style migrations)
- Over-engineering risk — a full framework for a tiny script is wasted complexity

Engineering judgement: a framework is an investment — pay the learning and lock-in cost only when the benefits outweigh it.

Frameworks vs Libraries

Aspect	Library	Framework
Control flow	Your code calls the library	Framework calls your code (IoC)
Scope	One focused task (HTTP, dates, charts)	Whole application structure
Architecture	You design it	Provided — conventions & structure
Flexibility	Mix and match freely	Commit to its rules & ecosystem
Replacement cost	Usually easy to swap	Expensive — often a rewrite
Examples	lodash, axios, NumPy, jQuery	ASP.NET Core, Angular, Flutter, Spring Boot

The decisive test: who is in control of the program's flow?

The Grey Zone – Where Do These Belong?

React	Calls itself a UI library — but its lifecycle controls your components, and React + Router + state libraries form a de-facto framework
jQuery	Pure library: you call $() whenever you choose; it never calls you
Express.js	Minimal web framework: it owns the request loop and calls your route handlers — even though it feels 'library-light'
Next.js / Angular / Flutter	Full frameworks: routing, build system, conventions, lifecycle — the platform is in charge

Checkpoint: is the .NET Base Class Library a framework or a library? Defend your answer.

Checkpoint 1 – Quick Thinking

1. In one sentence, what does Inversion of Control mean? Give one example from a tool you have used.

2. Your friend says: “I imported axios into my project, so I'm using a framework.” Correct or not? Why?

3. Name two benefits AND two costs of adopting a large framework like Angular for a 3-month student project.

Pair up — 3 minutes — then we discuss answers together.

PART 2 OF 4

Types of Frameworks & Industry Use

- The framework landscape — six major families
- Frontend, backend, mobile and testing frameworks
- How a full-stack architecture fits together
- Who uses what in industry — real examples

The Framework and Tool Landscape

Frontend (Web UI)

React • Angular • Vue • Next.js

Backend / Server-side

ASP.NET Core • Spring Boot • Express • Django

Mobile

Flutter • React Native • SwiftUI • Jetpack Compose

Testing & Quality

xUnit • JUnit • Jest • Selenium • Playwright

DevOps / CI-CD / Cloud

GitHub Actions • Docker • Kubernetes • Terraform

AI-assisted & Agentic

GitHub Copilot • Claude Code • LangChain • agents

Frontend Frameworks – Building Modern UIs

- Shared ideas: components, declarative UI, state management, client-side routing, virtual/reactive rendering

	React	Angular	Vue
Type	UI library + ecosystem	Full framework (batteries included)	Progressive framework
Language	JavaScript / TypeScript (JSX)	TypeScript	JavaScript / TypeScript
Best fit	Flexible SPAs, huge talent pool	Large enterprise apps, strict structure	Gentle learning curve, lightweight apps
Used by	Meta, Netflix, Airbnb	Google, Microsoft Office web	Alibaba, GitLab

This module: React (Lecture 02) — the most demanded frontend skill in the job market.

Backend Frameworks – APIs and Business Logic

- Role: expose REST APIs, run business rules, talk to databases, enforce security

Framework	Language	Strengths	Typical adopters
ASP.NET Core	C#	High performance, DI built in, enterprise tooling, cross-platform	Stack Overflow, banks, Microsoft
Spring Boot	Java	Mature ecosystem, microservices support	Netflix backend, enterprises
Express.js / NestJS	JavaScript / TS	Minimal & fast to start; same language as frontend	Uber, PayPal (Node)
Django / FastAPI	Python	Rapid development; FastAPI popular for AI/ML APIs	Instagram (Django)

This module: C# + ASP.NET Core (Lectures 03–04) — controllers, services, REST APIs, auth, database integration.

Mobile Frameworks – One Codebase or Two?

Native

- SwiftUI (iOS) • Jetpack Compose (Android)
- Best performance & full device API access
- Two codebases → double the team & cost

Cross-platform

- Flutter (Dart) • React Native (JS/TS)
- One codebase → iOS + Android (+ web/desktop)
- Flutter renders its own UI — consistent look, near-native speed

Industry examples: Google Pay & BMW apps use Flutter; Instagram and Shopify use React Native. Cross-platform now dominates startup and SME development.

This module: Flutter (Lectures 05–06).

Testing & Quality Frameworks

Level	Purpose	Frameworks / tools
Unit testing	Test one class/function in isolation	xUnit & NUnit (.NET) • JUnit (Java) • Jest (JS)
Integration / API	Test components working together	Postman/Newman • REST-assured • Supertest
End-to-end (UI)	Test full user journeys in a browser/app	Selenium • Playwright • Cypress • Flutter integration tests
Static quality	Analyse code without running it	SonarQube • ESLint • Roslyn analyzers

Why they are frameworks: the test runner discovers and calls YOUR test methods — Inversion of Control again. These tools plug into CI/CD pipelines (Lecture 07) so every commit is verified automatically.

How It All Fits – Full-Stack Architecture

Delivery layer: Git + GitHub → GitHub Actions CI/CD → Docker → Cloud hosting (Azure / AWS / Vercel)

AI-assisted tools (Copilot, Claude Code) accelerate every box in this diagram — Lectures 09–11.

Frameworks in Industry – Who Uses What

Company / product	Frameworks in production	Why it works for them
Netflix	React (UI) + Spring Boot microservices	Independent teams ship services on their own schedules
Stack Overflow	ASP.NET Core + SQL Server	Massive traffic on a famously small server footprint
Google Pay, BMW, Toyota	Flutter	One team, one codebase, consistent UX on both platforms
Airbnb, Meta	React (+ React Native history)	Reusable components across a very large product surface
Instagram	Django (Python)	Rapid iteration from startup to billions of users

Pattern: companies choose frameworks that match their scale, team structure and speed of change — not the newest or trendiest option.

Checkpoint 2 – Spot the Stack

- **Think of an app you used today (PickMe, WhatsApp, your bank, Daraz…)**
- **In pairs (4 minutes), sketch its likely architecture:**
  - Which frontend framework family? Web, mobile, or both?
  - What does the backend need to do? Which framework family fits?
  - Where is the data stored? What about payments or maps — frameworks or libraries?
- **Two pairs will present their guess — we'll critique it together as a class**

Goal: practise seeing every product as a set of framework decisions.

PART 3 OF 4

Framework Selection

- Selection criteria that professionals actually use
- A repeatable selection workflow
- Scoring candidates objectively
- Case study + your turn: a group selection activity

Framework Selection Criteria

Project fit

Does it solve THIS problem class well? (real-time, CRUD, mobile…)

Team skills

Current expertise beats theoretical superiority

Community & ecosystem

Docs, packages, Stack Overflow answers, hiring pool

Performance & scalability

Benchmarks under YOUR expected load, not marketing

Maturity & longevity

Release history, corporate backing, LTS versions

Security & compliance

Patch cadence, CVE history, audit requirements

Cost & licensing

Open-source licence terms, hosting cost, paid tiers

Integration

Plays well with your other frameworks, DBs, cloud

A Repeatable Selection Workflow

1. Requirements	2. Constraints	3. Shortlist	4. Prototype	5. Decide & record
Functional + non-functional (load, security, offline…)	Team skills, budget, deadline, existing systems	2–3 candidates per layer that fit the constraints	Build a small spike / proof-of-concept in each	Score, choose, document the rationale (ADR)

Architecture Decision Record (ADR): a one-page note capturing context, options considered, decision and consequences. Future maintainers (and your markers!) will thank you. You will write one for the main assignment.

Scoring Candidates – Weighted Decision Matrix

Example: choosing a frontend framework for a startup MVP (scores 1–5)

Criterion	Weight	React	Angular	Vue
Team familiarity	30%	4	2	3
Time to MVP	25%	4	3	5
Hiring pool (LK + remote)	20%	5	3	3
Ecosystem for our features	15%	5	4	4
Long-term maintainability	10%	4	5	4
Weighted total	**100%**	**4.35 ✓**	**3.05**	**3.85**

Weights encode YOUR context — a bank would weight security and maintainability, flipping the result.

When Framework Choices Go Wrong

Hype-driven choice	Adopting the newest framework for a critical system → immature ecosystem, breaking changes every month
Ignoring team skills	A Java team forced onto a Node stack → 6 months of slow, buggy delivery while 'learning on the job'
Longevity blindness	Building on AngularJS near its end-of-life (support ended 2022) → forced rewrite with zero new features
Over-engineering	Kubernetes + microservices for an app with 200 users → the team maintains infrastructure instead of features

Every failure above traces back to skipping a step in the selection workflow.

Case Study – “CampusEats” Food Delivery Startup

Brief: 3 developers, 4 months, limited budget. Needs: student-facing mobile app, restaurant web dashboard, orders + payments + live tracking, must scale if it succeeds.

Layer	Choice	Justification (criteria applied)
Mobile app	Flutter	One codebase for iOS + Android; 3-person team can't staff two native tracks
Web dashboard	React	Huge ecosystem (tables, charts); largest hiring pool when the team grows
Backend API	ASP.NET Core	One shared REST API for both clients; performance headroom; built-in auth & DI
Database	PostgreSQL / SQL Server	Relational fit for orders & payments; managed cloud hosting available
Delivery	GitHub Actions + Docker + Azure	Free tier CI/CD; containers keep dev = production

Notice: every choice cites a criterion, not a preference. This is the standard your assignment ADR must meet.

Group Activity – You Are the Tech Lead

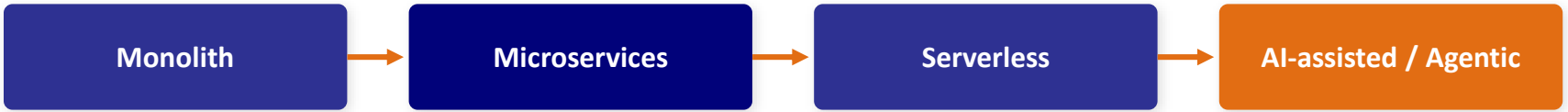
- **Form groups of 4 — 12 minutes**
- Your scenario (pick one):
  - A) Hospital appointment system for a private hospital chain (security & compliance heavy)
  - B) Real-time exam-hall seat finder for SLIIT (3-week deadline, web only)
  - C) E-commerce app for a clothing brand (mobile-first, must scale for sales seasons)
- Deliverable: stack choice for frontend / mobile / backend / database / hosting + top 3 criteria you weighted and WHY
- **Two groups present (2 minutes each); the class challenges one decision per group**

PART 4 OF 4

Emerging Trends in Frameworks

- From monoliths to microservices and serverless
- Cloud-native development and DevOps automation
- AI-assisted and agentic software development
- What these trends mean for YOUR framework choices

How Application Architecture Evolved



One deployable unit
Simple to start,
hard to scale teams

Many small services
Independent deploys,
operational complexity

Functions on demand
No servers to manage,
pay per execution

AI generates, tests
and reviews code with
human oversight

Key insight: newer ≠ better for every project. Each era added options, not replacements — monoliths still power excellent products (Stack Overflow!).

Trend 1 – Microservices Frameworks

- Idea: split the application into small, independently deployable services, each owning its data
  - Frameworks & tools: Spring Boot + Spring Cloud, ASP.NET Core microservices, Docker, Kubernetes for orchestration
  - Supporting patterns: API gateway, service discovery, message queues (RabbitMQ, Kafka)
- Wins: team autonomy, independent scaling, fault isolation, polyglot freedom
- Costs: distributed-system complexity, network failures, monitoring overhead, harder local dev
- Use when: large teams, different scaling needs per component — Netflix, Amazon, Uber scale

Rule of thumb: don't adopt microservices until a well-built monolith actually hurts.

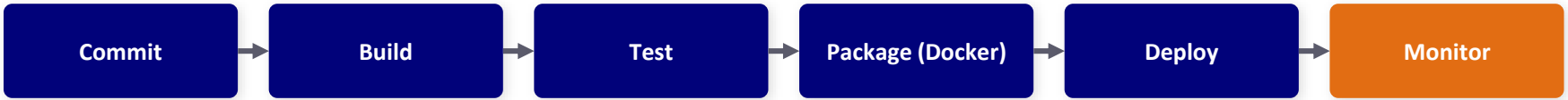
Trend 2 – Serverless Computing

- Idea: deploy functions, not servers — the cloud runs, scales and bills them per execution
  - Platforms: AWS Lambda, Azure Functions, Google Cloud Functions, Cloudflare Workers
  - Frameworks: Serverless Framework, AWS SAM/Amplify — define functions + triggers as code
- Great for: event-driven jobs (image resize, notifications), APIs with spiky traffic, scheduled tasks
- Watch out for: cold starts, execution time limits, vendor lock-in, tricky local testing
- Example: a CampusEats 'send order confirmation email' function — runs only when orders happen, costs cents

Serverless complements, not replaces, your main API — most real systems are hybrids.

Trend 3 – Cloud-Native & DevOps Automation

- Cloud-native: applications designed FOR the cloud — containers, declarative config, resilience, observability
  - Containers & orchestration: Docker packages the app + environment; Kubernetes runs it at scale
  - Infrastructure as Code: Terraform / Bicep — your servers are version-controlled text
- DevOps: culture + automation joining development and operations
  - CI/CD frameworks: GitHub Actions, GitLab CI, Jenkins, Azure DevOps



A push to GitHub can reach production in minutes — automatically tested at every step (Lectures 07–08).

Trend 4 – AI-Assisted & Agentic Development

AI-assisted coding

Copilots suggest code, tests and docs inside the IDE. Developer stays the author. Tools: GitHub Copilot, Claude, IDE assistants.

Agentic development

AI agents plan multi-step tasks: read the codebase, edit files, run tests, open pull requests. Tools: Claude Code, Copilot agents; frameworks like LangChain build custom agents.

Human-in-the-loop is non-negotiable:
engineers must review, test and take responsibility for AI-generated code. Risks: subtle bugs, security flaws, licence issues, over-reliance. Your value shifts toward requirements, architecture, evaluation — exactly LO1 and LO4.

What These Trends Mean for Your Choices

- Frameworks now compete on cloud & AI readiness — containers, serverless targets, AI tooling support
- Deployment is part of the stack decision: a framework without good CI/CD and hosting story costs you later
- Prefer frameworks with strong conventions — AI tools generate better code for well-documented, conventional frameworks
- Architecture flexibility matters: choose stacks that can start monolith and split into services later
- Evergreen skill: the evaluation method survives every trend — criteria, trade-offs, evidence, decision records

Frameworks will change every five years. Your selection method shouldn't.

Quick Quiz – Test Yourself

Q1. The defining difference between a framework and a library is: (a) size (b) language (c) who controls the flow of execution (d) licence

Q2. “Don't call us, we'll call you” describes: (a) REST (b) Inversion of Control (c) CI/CD (d) serverless

Q3. A 3-person startup needs iOS + Android apps in 4 months. The strongest first candidate is: (a) two native apps (b) Flutter (c) desktop app (d) static website

Q4. Which is the BEST reason to pick framework X? (a) trending on social media (b) used by Google (c) highest weighted score on your project's criteria (d) newest release

Q5. In agentic AI development, the human's essential role is: (a) typing faster (b) reviewing, testing & taking responsibility (c) avoiding documentation (d) none

Answer individually first — then we vote by show of hands.

Quiz Answers & Why

Q	Answer	Why
1	(c)	Control flow defines the relationship — IoC is the framework's signature
2	(b)	The Hollywood Principle is the classic phrasing of Inversion of Control
3	(b)	One codebase fits the team size and deadline; native doubles the work
4	(c)	Selection must trace to weighted project criteria, not popularity or novelty
5	(b)	Human-in-the-loop review and accountability are non-negotiable

Scored 4–5? You've met today's outcomes. Below 3? Revisit Parts 1 and 3 before next week.

Discussion – Take a Position

1. “In five years, AI agents will choose our frameworks for us.” Agree or disagree — what part of selection is hardest to automate?

2. Should a university teach the MOST POPULAR frameworks or the BEST-DESIGNED ones? Are they the same thing?

3. Your company's 8-year-old framework is stable but unfashionable, and juniors don't want to learn it. Rewrite, wrap, or stay? Defend your answer with criteria.

No marks, no wrong answers — only undefended ones.

Summary – What You Should Remember

- **A framework is a semi-complete application: it provides structure and calls YOUR code (Inversion of Control)**
- Frameworks trade learning curve and lock-in for productivity, quality, security and team scalability
- Library vs framework = who controls the flow of execution
- Six framework families: frontend, backend, mobile, testing, DevOps/cloud, AI-assisted — real systems combine them
- Selection is a method: requirements → constraints → shortlist → prototype → decide & record (ADR + weighted matrix)
- **Trends (microservices, serverless, cloud-native, agentic AI) change the options — never the method**

Next Week & How to Prepare

Lecture 02 — Advanced React Frontend Development: components, props & state, hooks, events, forms & validation, routing.

- **Before next week:**
  - Install Node.js (LTS) and VS Code; verify with `node -v` and `npm -v`
  - Refresh JavaScript ES6: arrow functions, destructuring, map/filter, modules
  - Skim: Banks & Porcello, Modern Patterns for Developing React Apps — Ch. 1–2
- **Lab connection:**
  - Lab 01 this week: environment setup + scaffold and explore your first React app (counts toward the 10% lab submission)

Learning Outcome Mapping & References

Lecture section	LO1 Evaluate	LO4 Select & justify
Fundamentals, benefits & costs, frameworks vs libraries	●	
Framework types, architecture & industry use	●	○
Selection criteria, workflow, matrix, case study & activity	○	●
Emerging trends (microservices, serverless, cloud-native, agentic AI)	●	●

References

- Banks, A. & Porcello, E. (2020) Modern Patterns for Developing React Apps. 2nd edn. O'Reilly.
- Rose, R. (2022) Flutter and Dart Cookbook. O'Reilly.
- Ford, N., Richards, M., Sadalage, P. & Dehghani, Z. (2021) Software Architecture: The Hard Parts. O'Reilly.
- Biswas, A. & Talukdar, W. (2025) Building Agentic AI Systems. Packt.
- Official docs: react.dev • learn.microsoft.com/aspnet/core • docs.flutter.dev • docs.github.com/actions